



PROMPT ENGINEERING

Prompt Engineering & Reasoning Patterns

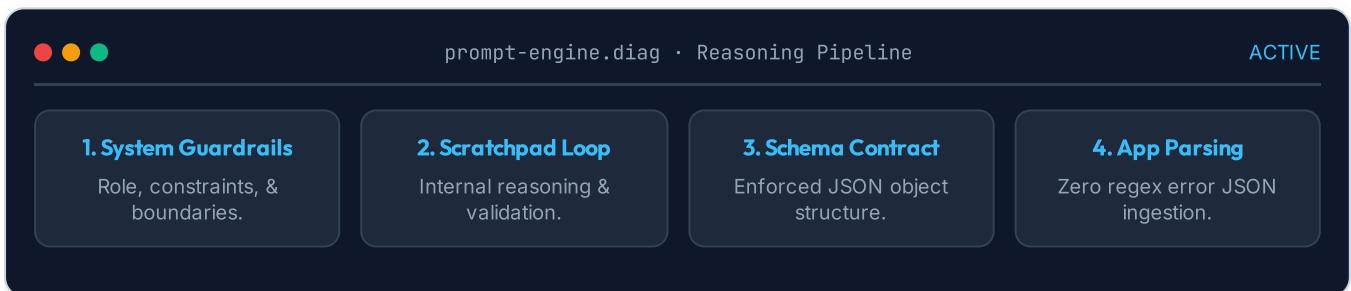
Production reasoning patterns, chain-of-thought frameworks, and guaranteed JSON schema decoding.

EXECUTIVE SUMMARY

Reliable AI software requires eliminating non-deterministic outputs. This handbook details structured chain-of-thought prompts, schema constraints, and prompt caching strategies that reduce token expenses by up to 80%.

1 Reasoning Frameworks & Deterministic Prompts

By instructing models to plan before acting through explicit scratchpad sections, hallucination rates drop dramatically. Constrained decoding guarantees that every response matches your application data types.



2 Production Implementation & Schema

```
// Schema-Constrained Prompt Contract (TypeScript)
const response = await ai.chat({
  messages: [{ role: 'user', content: query }],
  response_format: { type: 'json_object' },
  schema: z.object({
    findings: z.array(z.string()),
    confidenceScore: z.number().min(0).max(1)
  })
});
```



3 Prompt Caching for 80% Cost Reduction

Place static documentation, schemas, and extensive system rules at the top of your prompt prefix. Platforms like Anthropic and Google cache these tokens, cutting latency and billing on subsequent calls.

```
prompt-eval · Output Verification CONNECTED

[EVAL] Executing test suite across 100 benchmark queries...
✓ [SCHEMA] 100/100 outputs parsed cleanly into validated JSON
✓ [CACHE] 84.6% cache hit rate on static system prefix tokens
✓ [LATENCY] Time-to-first-token reduced from 920ms to 240ms
[COST] Effective API expenditure reduced by 78.2%
```

4 Key Takeaways & Best Practices

✓ Separate Reasoning

Allow models a private thinking scratchpad before demanding the final JSON object to ensure accurate logic.

✓ Enforce JSON Schemas

Never parse LLM responses with custom regexes; always enforce strict JSON mode validated by Zod or Pydantic.

✓ Prefix Stability

Keep system prompt prefixes completely stable across calls to maximize provider cache hit rates.

✓ Negative Constraints

Explicitly define what the model must NOT do (e.g. "Do not assume missing parameters; ask first").

IMPLEMENTATION SUPPORT

Need engineering assistance integrating these architectures into your business? Contact our team directly at futureai4all@gmail.com.